Simulated Annealing: A Short Primer

Background reading for the accompanying paper in this repository. Written for a mixed audience; only basic probability is assumed.

The problem: rugged landscapes

Many optimization problems reduce to minimizing a cost function E(x) — sometimes called an energy, by analogy we will justify shortly — over a space of candidate solutions x. If E is smooth and bowl-shaped, gradient descent solves it: always step downhill, arrive at the bottom. The difficulty begins when the landscape is *rugged*: many local minima separated by barriers. A pure downhill method stops at the first valley it enters, which is usually not the deepest one. Combinatorial problems (routing, scheduling, layout, component placement) and many fitting problems (multi-peak spectral fits, sensor placement, alignment searches) have exactly this character, and exhaustive search is hopeless because the number of candidates grows exponentially with problem size.

Simulated annealing (SA), introduced by Kirkpatrick, Gelatt, and Vecchi in 1983, attacks rugged landscapes with a strategy borrowed from metallurgy. When a metal is cooled slowly — annealed — its atoms have time to escape strained, high-energy arrangements and settle into a low-energy crystal. Quench it rapidly and it freezes into a defect-riddled state: a local minimum. SA translates this directly: allow the search to sometimes move *uphill*, generously at first, then ever more reluctantly as a control parameter — the temperature — is lowered.

The algorithm

SA needs four ingredients: a **state representation** x, a **cost function** E(x), a **move set** that proposes a random neighbor x′ of the current state (a small perturbation), and a **cooling schedule** T(k) that decreases over iterations. The loop is short. From current state x, propose x′ and compute $\Delta E = E(x') - E(x)$. Then apply the **Metropolis criterion**:

$$\text{accept } x' \text{ with probability } p = 1 \text{ if } \Delta E \le 0, \text{ otherwise } p = \exp(-\Delta E/T)$$

Downhill moves are always taken. Uphill moves are taken with a probability that shrinks exponentially with the size of the cost increase and with falling temperature. At high T nearly everything is accepted and the search wanders freely across the landscape, hopping barriers at will. At low T only improvements survive and the algorithm behaves like local descent, refining whichever basin it occupies. The slow passage between those regimes is where the work gets done: broad exploration first, commitment later. Figure 1 shows the acceptance rule at several temperatures.

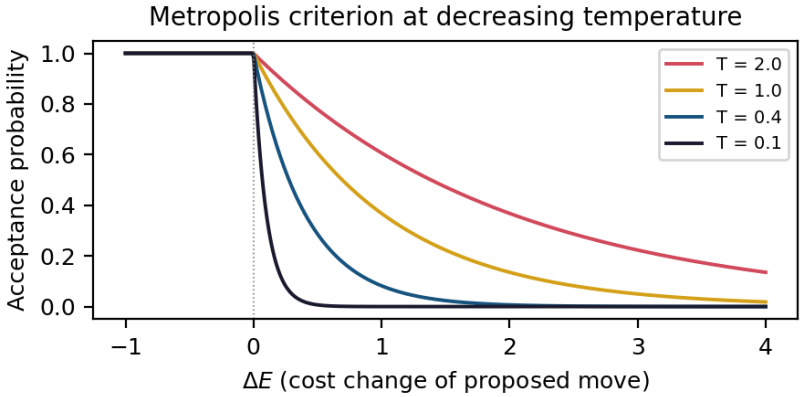


Figure 1. Metropolis acceptance probability versus proposed cost change ΔE. Downhill (ΔE ≤ 0) is always accepted. As T falls, uphill tolerance collapses and the search hardens from random exploration into strict descent.

The physical analogy is not decoration. The Metropolis rule is the same sampling rule used in statistical mechanics to draw states from the Boltzmann distribution $p(x) \propto \exp(-E(x)/T)$; at fixed T, the algorithm equilibrates toward that distribution, which concentrates on low-energy states as $T \to 0$. Cooled sufficiently slowly, SA provably converges to the global minimum — though the provable schedule is far slower than anything

practical. Real implementations use geometric cooling, $T_{k+1} = \alpha T_k$ with α typically 0.9–0.999, and accept that the result is a very good minimum rather than a certified global one.

A worked example

Figure 2 applies SA to a deliberately rugged one-dimensional cost function with several local minima. The run starts at $x \approx 5.2$, far from the global minimum, with T■ = 4 and geometric cooling ($\alpha = 0.995$, 1500 steps; proposals are Gaussian steps of width 0.5). Panel (a) colors each visited state by the temperature at which it was visited: hot samples (red) are scattered across every basin, including high ridges a greedy method could never cross; cold samples (blue) cluster tightly in the deepest valley, where the run terminates near $x \approx -0.59$. Panel (b) shows the energy trace against the cooling schedule — noisy and exploratory early, then settling as uphill moves are progressively refused. Note that the energy is *not* monotone: the temporary increases early in the run are not a malfunction but the entire mechanism of escape.

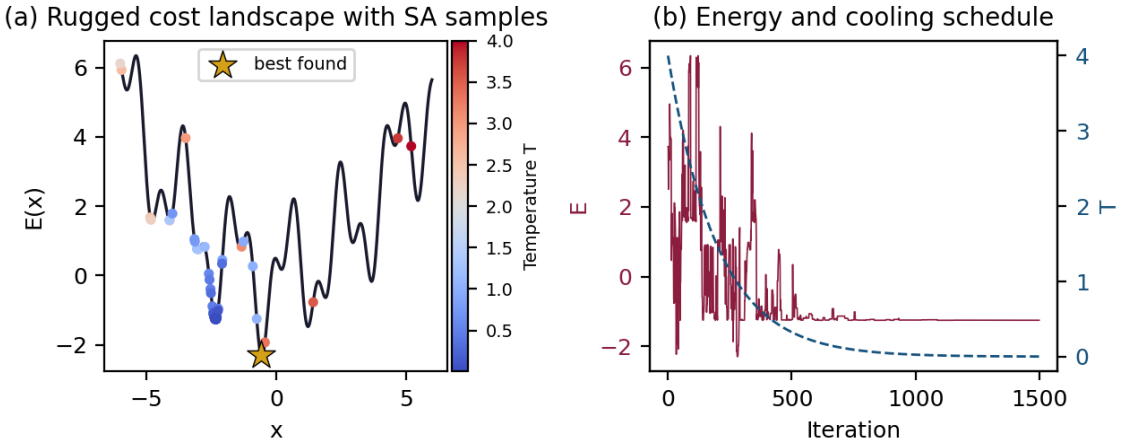


Figure 2. (a) SA on a rugged 1-D landscape; points are visited states colored by temperature, the star marks the best state found (the global minimum). (b) Current energy (red) and temperature (dashed blue) versus iteration.

Practical notes

Performance depends far more on the **move set** and **schedule** than on any other detail. Moves should be local enough that ΔE is usually modest — if every proposal is essentially a random restart, the Metropolis rule degenerates into blind search — yet rich enough to reach the whole space. A useful tuning heuristic is to set T■ so that roughly 80–95% of early moves are accepted, and to cool slowly enough that acceptance declines gradually rather than collapsing. Track and store the *best state ever visited* separately from the current state; the current state at termination is not guaranteed to be the best one seen. Because runs are stochastic, several independent runs from different seeds are cheap insurance and give a variance estimate for free. Common variants include restarts (reheat when progress stalls), adaptive schedules keyed to acceptance rate, and parallel tempering, which runs several temperatures simultaneously and swaps states between them.

SA's virtues are its indifference to the cost function's form — no gradients, no convexity, no continuity required; E(x) may be a lookup, a simulation, or a measurement — and the smallness of its implementation, a few dozen lines. Its cost is the price of that generality: many function evaluations and problem-specific tuning. The accompanying paper in this repository picks up from here, treating the specific formulation, move set, and schedule used in this software; the interactive annealing explorer lets you vary T■ and α and watch the exploration-to-commitment transition of Figure 2 unfold live.